Measuring LLM Security Verdicts: Coverage, Abstention, and Adversarial Suppression

Author Name
Independent Researcher
Email: author@example.org

Abstract—Large language models (LLMs) are increasingly used as verifiers of security findings: a static analyzer proposes a candidate and a model adjudicates whether it is real. Whether such a verdict can be trusted is rarely measured. We adapt the four-way decomposition—an instrument previously used to show that static analyzers flag constructs rather than vulnerabilities—to LLM verifiers and extend it with evidence gating, an adversarial comment attack, a composition study, and a calibration analysis. On a 21-pair real-fix security subset, an evidence-grounded verifier tracks the fix far more often than the pattern matcher it is meant to replace (30.5%, 95% CI [22.9,37.1], vs. 4.8%) and does not over-report, but it abstains on 49% of judgements and misses 69% of genuine fixes. Its bottleneck is therefore coverage, not accuracy. On a labelled benchmark the verifier does not improve on the analyzer's precision (at most 2 of 15 false positives rejected), and a cross-language check confirms the verdict itself is verifier-dependent. Adversarial comments evade 26% (fixed templates) to 50% (an adaptive attacker), and nearly every successful evasion is an abstention rather than a false positive: an attacker need only make the verifier unsure, not wrong. Composing the LLM with a weaker deterministic verifier relocates trust rather than adding it—grounding a claim helps at no recall cost, while requiring consensus loses recall for no gain. Our scaled evidence is a single 21-pair subset with one model family at k=5, and we state this plainly. We release a reproducible harness and argue that abstention and coverage should be reported as first-class outcomes of security agents.

Index Terms—abstention, adversarial robustness, calibration, code property graphs, evidence grounding, large language models, program analysis, selective prediction, static analysis, vulnerability verification.

I. Introduction

Automated vulnerability analysis has converged on a standard shape: a policy queries tools and a code graph and is trained or prompted to report findings [1]–[4]. LLM agents now drive that loop—reasoning over repositories, invoking analyzers, and drafting fixes. In each case a model makes verification judgements: given a candidate produced by a scanner, a fuzzer, or another model, is it real? As agents take over the gate between a warning and a fix, the question becomes whether their verdicts can be trusted.

That question is harder than it looks. A model can be right about the code and wrong about the vulnerability: it can flag a dangerous-looking construct that the patch did not touch, agree that a benchmark sample is vulnerable without reasoning about the vulnerability, or become confident on inputs unlike its training distribution. For a static analyzer, the companion study behind this work showed that the fraction of commits whose verdict moves with a fix can be two orders of magnitude below its benchmark true-positive rate, and introduced the four-way decomposition to make that gap visible [5]. We ask the same of LLM verifiers.

The stakes are concrete. Static analysis has always lived with false positives [6], and machine-learning vulnerability detectors inherit the same label and validity problems [7]–[9]. When a model is placed in front of remediation, two failures are possible: it can confirm a non-vulnerability (wasted effort, eroded trust) or, more dangerously, suppress a real one. Prior work shows that LLM security detectors are easily steered by adversarial comments [10] and by indirect prompt injection [11]; what has not been measured systematically is whether an LLM verdict tracks the vulnerability at all, how it fails when it does not, and how it behaves under composition and an adversary.

Contributions.

- An instrument. We apply the four-way decomposition to LLM verifiers, gate every verdict on machine-checkable evidence, and report the coverage of the process. The harness, corpora, and pinned analyzers are released.
- A coverage-bound finding. An evidence-grounded verifier tracks the fix far more often than a pattern matcher (30.5%, 95% CI [22.9,37.1], vs. 4.8%) and does not over-report (1.0× vs. 6.0×), but abstains on 49% of judgements and misses 69% of genuine fixes. Its bottleneck is silence, not error—a profile that precision-only evaluation cannot see.
- Abstention as the attack surface. Under adversarial comments, almost every successful evasion is an abstention rather than a false positive; an adaptive (LLM-authored, iterative) attacker reaches 50% evasion against 26% for fixed templates.
- Composition relocates trust. Evidence grounding removes an ungrounded positive at no recall cost; requiring agreement with a weaker deterministic verifier loses recall and gains nothing.
- A negative result on labelled ground truth. On OWASP Benchmark (2,740 labelled cases) the an-

alyzer reaches recall 0.905 / precision 0.734, but the LLM verifier does not improve precision: in a matched sample neither a local 7B nor a cloud model rejects false positives (0–2 of 15), and blinding the scanner hint only raises abstention. Verification is coverage-bound.
- Calibration and cost. The verifier is precise (0% fixed-side false positives in every repeat) but not reliably calibrated (ECE 0.07–0.35 across repeats); disabling model thinking cuts cost ∼8× at a measured precision cost, and a content-addressed verdict cache removes 100% of repeat cost.

We are explicit that the scaled evidence is small (Section VII) and treat the paper as a measurement-methodology contribution rather than a claim of a large effect.

II. Background and Related Work

A. LLM agents for vulnerability analysis

A fast-moving line of systems couples LLMs with static analysis, symbolic execution, and fuzzing. OpenAnt decomposes code and adversarially verifies candidates [4], and agentic pipelines such as VulAgentRL, Antares, and EAVA ground discovery in analyzers and code graphs [1]–[3]. These systems report improved discovery, but they generally do not measure whether a verdict tracks the vulnerability, and they rarely report the coverage–precision trade-off of the verifier itself. We measure the verdict rather than the discovery rate.

B. Verifiers and verifiable rewards

VulAgentRL uses a code property graph both as a tool and as a reward verifier with exact node matching [1]; Antares trains small models with verifiable rewards [2]; EAVA couples supervised tuning with reinforcement learning and requires evidence for a verdict [3]. "Verifiable" here means that the training signal can be checked, not that the model's inference-time verdict is grounded. We do not train; we ask what trust a grounded verdict earns at inference time and how it fails.

C. Adversarial robustness of LLM code analysis

ALIBI shows that adversarial source comments defeat LLM vulnerability detectors with high success rates [10]; indirect prompt injection compromises tool-integrated agents [11]; and code security-hardening work demonstrates both the risk and partial defenses [12]. Earlier work established that AI code assistants frequently produce insecure code [13], [14]. We attack the verifier rather than the detector, split evasion by outcome (false positive vs. abstention), and include an adaptive attacker.

D. Abstention, selective prediction, and calibration

Selective classification makes rejection an explicit learning problem [15], and out-of-distribution detection is the closely related recognition task [16]. Modern neural networks are often miscalibrated [17], and abstention has become a first-class topic for LLMs, where models can partially "know what they know" [18]; recent surveys catalogue the design space [19]. We bring this lens to security verification and show that abstention is not merely a safety valve—it is the dominant adversarial target and the verifier's main bottleneck.

E. LLM-as-a-judge reliability

Using a model to adjudicate another model's output is now standard [20], [21], and its failure modes are documented: evaluators are order- and format-sensitive [22] and favour their own generations [23]. Security verification is a judge problem in this sense, but the ground truth is a code change rather than a preference; we exploit that difference by scoring verdicts against the fix.

F. Static analysis false positives and measurement validity

Static analysis has been deployed at scale [6], and its false positives are a persistent obstacle in practice. Machine learning for security inherits methodological pitfalls—leakage, temporal bias, and ill-defined labels [9]—and vulnerability datasets in particular are noisy [8]. Classical and learned vulnerability detectors and their limits are well studied [7], [24], [25]. The OWASP Benchmark is the standard scanner yardstick [26]; the companion study showed its score is not well defined and does not predict real detection, and introduced the four-way decomposition [5]. We extend [5] from static tools to LLM verifiers.

G. Code property graphs and query engines

Program-analysis tooling exposes relational views of code: code property graphs [27] and object-oriented query languages over relational data [28] are the basis of modern taint engines. Our evidence gating uses a lightweight tree-sitter graph rather than a sound dataflow engine; Section VII treats the resulting weakness explicitly.

H. Ensembles and composition

Combining predictors is a classical idea [29], and diversity is the property that makes an ensemble work [30]. Our composition result is a negative instance of that principle in a security setting: adding a weaker verifier by consensus does not add diversity, it adds the weaker verifier's blind spot.

III. Method

Figure 1 shows the pipeline end to end. A revision pair is scanned by a pinned analyzer; each candidate becomes a bounded verdict request; the verifier must ground its verdict in resolvable evidence; the vulnerable and fixed verdicts are then decomposed and summarised. An adversary can inject comments before verification.

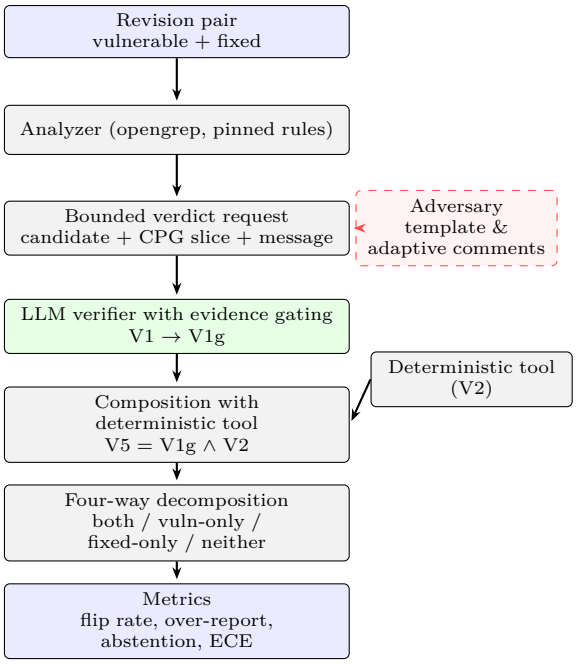


Fig. 1. RewardAudit architecture. A revision pair is scanned by a pinned analyzer; each candidate is turned into a bounded verdict request over a tree-sitter code property graph and the commit message. The verifier must ground its verdict in resolvable evidence (V1→V1g). An adversary (dashed) can inject comments before verification, and a deterministic tool can be composed with the verifier. Verdicts on the two revisions feed the four-way decomposition, which yields the reported metrics.

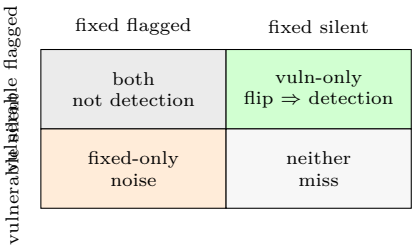


Fig. 2. The four-way decomposition over a revision pair. Only vuln-only is detection; both means the verifier flagged a construct present before and after the fix.

A. The four-way decomposition

For a vulnerable/fixed revision pair, we classify a verifier's verdict into one of four cells (Fig. 2). Only vuln-only is evidence of detection; both means the verifier flagged a construct present before and after the fix, so its verdict could not have been about the vulnerability. From the cells we derive the flip rate (vuln-only fraction), the naive coverage (fraction flagged on the vulnerable revision), and their ratio, the over-report factor.

B. Verifier arms

We compare four arms on identical inputs. V1 (LLM, ungated): the model's verdict is taken as-is. V1g (LLM, gated): every cited evidence reference must resolve to a real syntax node, or a positive verdict is downgraded to unknown. V2 (tool): the pinned analyzer's verdict. V5 (hybrid): flagged only if the gated LLM and the tool agree.

C. Bounded verdict request and evidence gating

The model never sees a repository. For a candidate it receives a bounded request: the candidate (rule, location, CWE hint), the enclosing function extracted from a tree-sitter code property graph, and the commit message as legitimate context. It must respond with a strict JSON verdict, cite path:line references, and may answer unknown_risk. Each reference is resolved against the graph; an unverifiable positive is downgraded. We deliberately never feed whole files by default, since that is the strawman this measurement is meant to beat.

D. Adversary

Templates: five plausible comment strategies (authority, sanitizer, tool-output, test-passed, reviewed) inserted just above the suspected sink. Adaptive: an attacker model writes a comment, observes the verifier's verdict, and refines over up to three rounds [10]. We split evasion into false-positive and abstention outcomes, because they target different failure modes and demand different defenses.

E. Calibration, selective risk, and cost

Per revision judgement we record confidence, correctness, and disposition. We compute expected calibration error (ECE), reliability bins, and a selective-risk/coverage curve [15], plus false-negative rate on vulnerable revisions and false-positive rate on fixed revisions. Cost is reported in tokens and, at known prices, dollars per true positive.

F. A worked example

Consider an analyzer warning on os.system(command) in a helper reached from an HTTP handler that reads an environment variable. An ungated model may answer true positive with a plausible but unfalsifiable rationale. Under V1g it must cite a graph node for the source and the sink; if the cited nodes do not exist or the call path is not present, the positive is downgraded to unknown_risk. This is the mechanism that produces the coverage–precision profile in Section V: grounding removes unverifiable positives, but a model that cannot produce verifiable evidence abstains, and abstention is scored as a miss on a fixed-side test.

IV. Experimental Setup

Corpora. (i) 21 Java security-fix pairs drawn from JavaVFC [31] (784 commits; 327 usable single-file; 207 materialised offline; the strong-security subset used here is 21). (ii) 5 hand-built Python vulnerable/fixed pairs and 3 canaries. (iii) The prior study's 21-file Java control set (18 vulnerable, 3 safe) [5]. Each revision is presented as an isolated file; the commit message is supplied as candidate context, as a bug report would be.

TABLE I
Cross-family verdicts on 5 Python pairs / 3 canaries.

Model	flip rate	both	canary FP
gpt-oss-120b	80%	1	1/3
gpt-oss-20b	100%	0	1/3
qwen3.8-27b	100%	0	0/3
deepseek-flash	100%	0	0/3
deepseek-v4-pro	80%	0	0/3 (1 abstain)

Analyzers. opengrep 1.30.0 with the prior study's pinned 690-rule Java ruleset (sha256 77df32e0…) and a minimal 5-rule Python ruleset.

Models. Five families via an OpenAI-compatible API: gpt-oss-120b, gpt-oss-20b, qwen3.8-27b, deepseek-flash, deepseek-v4-pro. The scaled JavaVFC run uses deepseek-flash (k=5, verdict cache disabled); the remaining families are checked on the 5-pair Python set.

Metrics and statistics. For the scaled arm we report the mean over k=5 independent runs with 95% bootstrap intervals; because reasoning models are non-deterministic even at temperature 0, we report the observed range alongside the interval. We use the exact McNemar test for paired comparisons on the same pairs.

Reproducibility. All assets are pinned by hash (pins.json); the control reproduction and the Python-pair measurement run fully offline; 125 unit and integration tests accompany the harness.

V. Results

A. Verifier dependence

On identical inputs, models disagree in both flip rate and failure mode; the cross-language check is summarised in Table I. gpt-oss-120b flipped on 4/5 pairs and flagged a safe canary; gpt-oss-20b and qwen and deepseek-flash flipped on 5/5 with one family showing a canary false positive; deepseek-v4-pro abstained on one vulnerable revision. The result is verifier-dependent—the phenomenon the instrument is designed to expose. The same holds at scale: within the single family we repeated five times, the flip rate spans 19.0–42.9% across runs (Section VII), so even one model does not have a stable point value.

B. LLM versus pattern matcher, and composition

The tool is the weakest arm: it flags the same construct on both revisions in 5 of 21 pairs, so it over-reports 6.0× its flip rate (Table II). The evidence-gated LLM tracks the fix far more often—30.5% [22.9,37.1] over five runs (E0) vs. the tool's 4.8%—and, unlike the tool, never over-reports (0 both in every run). In a single-run ablation, removing evidence gating admits one both (an ungrounded positive), and requiring agreement with the tool (V5) loses a flip without removing any both: the weaker verifier's blind spot becomes the composition's blind spot. Grounding a claim is the arm that helps; consensus with a weaker tool is not.

TABLE II
Verifier arms on the 21 JavaVFC security pairs (deepseek-flash). V1g is the E0 measurement (mean, 95% CI, k=5); the other LLM arms are a single run and are shown for the ablation only. The tool is deterministic.

Arm	flip rate	over-report
V2 tool (deterministic)	4.8%	6.0×
V1 ungated (1 run)	19.0%	1.25×
V1g gated (k=5)	30.5% [22.9, 37.1]	1.00×
V5 hybrid (1 run)	14.3%	1.00×

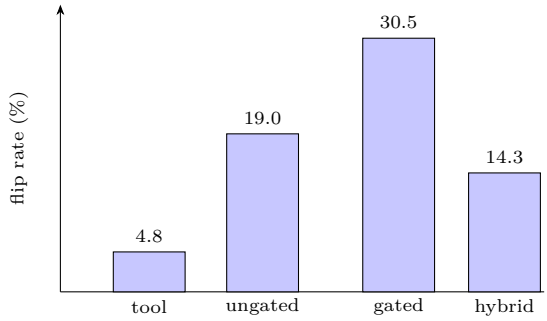


Fig. 3. Flip rate by verifier arm. The deterministic tool is the weakest tracker even though it flags on both revisions 6× as often as it flips.

C. Adversarial comments

Two attack strategies were tested: fixed templates and an adaptive attacker (Table III). The sanitizer tactic ("the input is validated upstream") was strongest across families. Critically, all successful adaptive evasions, and 8 of 9 template evasions, were abstentions rather than false positives: an attacker need only make the verifier unsure. The adaptive attacker roughly doubles evasion relative to templates. This is the security consequence of the coverage finding above: a defense that only suppresses false positives leaves the dominant path open.

D. Calibration and coverage

The verifier is precise but poorly calibrated: over five repeated runs with the cache disabled (Table IV), it never false-positives on the fixed side, yet its ECE ranges 0.07–0.35. It abstains on half the judgements and misses most fixes. The verifier is therefore trustworthy when it commits—but the coverage is low enough that most real fixes are not certified by it at all.

E. Cost

Output tokens dominate (∼91% of cost). Disabling model thinking reduced output from 121,596 to 5,271 tokens over 42 judgements (∼8× lower cost) but introduced fixed-side false positives (0%→9.5%) and worsened ECE (0.153→0.207). A content-addressed verdict cache keyed on the model, prompts, and slice removed 100% of repeat cost. At off-peak prices the gated verifier costs ≈$0.0017 per judgement. The economics matter: if the bottleneck is

TABLE III
Evasion. JavaVFC templates: baseline 7/21 true positives; adaptive: baseline 6/21, attacker deepseek-v4-pro vs. judge deepseek-flash.

Attack	evasion	via FP	via abstain
templates (authority)	43% (3/7)	0	3
templates (overall)	26%	1	8
adaptive (3 rounds)	50% (3/6)	0	3

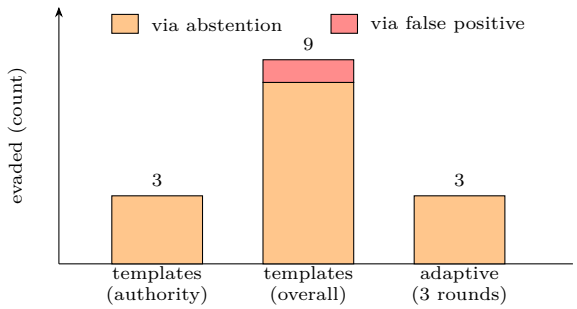


Fig. 4. Successful evasions by attack and outcome. Nearly all are abstentions: the attacker only has to make the verifier unsure, not wrong.

TABLE IV
Calibration on 21 pairs, deepseek-flash, k=5 mean [95% CI].

Metric	Mean [95% CI]
abstention rate	49.0 [44.3, 53.8]
detection (recall on vuln revision)	31.4 [27.6, 35.2]
false-negative rate	68.6 [64.8, 72.4]
fixed-side false-positive rate	0.0 [0.0, 0.0]
accuracy (scored)	73.5 [66.1, 81.8]
ECE	0.206 [0.128, 0.293]

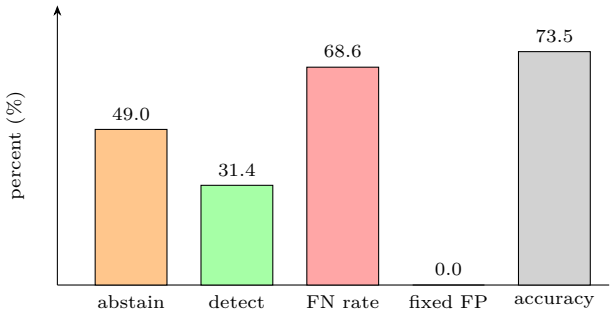


Fig. 5. Calibration and coverage (mean over five runs). The verifier is precise (0% fixed-side false positives) yet abstains on half the inputs and misses most fixes; ECE is 0.206 [0.128, 0.293].

coverage, the productive investment is better context and better evidence, not more sampling of the same prompt.

F. Detection effectiveness on a labelled benchmark

To test the whole pipeline against ground truth we ran it on the OWASP Benchmark for Java [26]: 2,740 labelled test cases (1,415 vulnerable, 1,325 safe). The pinned analyzer alone is a strong detector but over-reports (Table V).

We then asked whether the LLM verifier improves on that precision. On a matched sample (15 analyzer true positives, 15 analyzer false positives) we ran the verifier with and without the scanner's rule, message, and CWE hint ("blind"), to separate genuine judgement from echoing the scanner (Table VI).

Two things follow. First, no configuration rejects false positives: neither a local 7B nor the cloud model separates the safe near-misses from the vulnerable cases, and the analyzer's precision is not improved. Second, blinding does not help—removing the scanner's hint does not raise rejection, it drives abstention to 50–73% and cuts recall. The failure is therefore not scanner-confirmation bias but the model's inability to ground the deciding evidence (a sanitizer) from the bounded slice. Error gating aside, verification here is coverage-bound and adds nothing over the analyzer.

VI. Discussion

The bottleneck is coverage. Across the scaled run the verifier never false-positives on the fixed side and abstains on half of all judgements, but its confidence is not reliably calibrated (ECE 0.07–0.35) and it misses most fixes. Optimising a security agent for precision alone would miss this entirely. We argue abstention should be reported as a first-class outcome alongside precision and recall, and that a verifier which silently declines most inputs is not "accurate" in any deployment sense.

Attacks target abstention. Because the verifier already abstains when evidence is thin, an adversary does not need to convince it a vulnerability is safe—only to make it unsure. This reframes adversarial robustness for verifiers: defenses that only suppress false positives leave the dominant path open.

Composition relocates trust. Grounding a claim in verifiable evidence is cheap and strictly helpful here. Requiring consensus with a weaker verifier is not: it inherits that verifier's blind spots. "Add another verifier" is not a reliability strategy; "make the existing verdict answer for itself" is.

Design implications. Three rules follow. (i) Score the fix, not the sample: a verdict should be evaluated by whether it moves with a code change, which is cheap to collect and hard to game. (ii) Make abstention visible: agents should surface the fraction of findings they declined to decide, because that is where adversarial exploitation concentrates. (iii) Prefer grounding to consensus: require a verdict to answer for itself with resolvable evidence rather than adding another model whose agreement may only import a shared blind spot. A stronger form of grounding would require a machine-checkable certificate for every accepted verdict, but our results suggest the immediate constraint is coverage, not the strength of the evidence a model can already produce.

TABLE V
Analyzer on 2,740 labelled OWASP Benchmark cases.

Metric	Value
TP / FP / FN / TN	1,280 / 463 / 135 / 862
recall (TPR)	0.905
precision	0.734
false-positive rate	0.349

TABLE VI
LLM verification on the matched Benchmark sample. A cache hit or a blind run cannot echo the scanner's CWE hint.

Verifier	TP kept	FP rejected	abstain	prec.
local 7B, hinted	14/15	0/15	1/30	0.48
local 7B, blind	7/15	0/15	15/30	0.47
cloud, hinted	10/15	2/15	13/30	0.67
cloud, blind	5/15	0/15	22/30	0.62

A practical pitfall. A reasoning-model verifier silently degrades to "always abstain" when the token budget is consumed by reasoning tokens (finish_reason=length, empty content). Without the decomposition this is indistinguishable from a null result. We report it because it cost us a run.

VII. Threats to Validity

Scale. The scaled result is 21 security pairs; the cross-language check is 5 pairs, so the cross-family comparison is directional. Labels. JavaVFC is a screening corpus: a companion audit [5] found only ∼18% of it is a genuine security fix, so recall is a floor against a cleaner label set. Input shape. Each revision is an isolated file, which is a floor for tools that resolve types and near-neutral for pattern matchers. Evidence graph. Our gating uses a tree-sitter graph, not a sound dataflow engine [27], [28]; it can confirm that a cited path:line exists, not that taint flows. Non-determinism. Reasoning models vary run to run at temperature 0; we observed detection ranging 23.8–38.1% and flip rate 19.0–42.9% across five repeats of one configuration; the k=5 bootstrap intervals are narrower than that observed range and are reported alongside it. Single family at scale. Only deepseek-flash was repeated on the scaled corpus (k=5); the other four families are checked only at n=5, so "verifier-dependent" is established across vendors only on the small set. Contamination. Public fixes may appear in pretraining data; a temporal or private holdout is not yet used. Benchmark. OWASP Benchmark labels are synthetic and single-method, and the verifier ablation is a 30-case sample; it tests precision behaviour, not recall. Adversary. Our templates are fixed; a single adaptive attacker was used, and its adaptivity is shallow.

VIII. Ethics and Responsible Release

The adversarial comment generator is dual-use. This work is defensive: its purpose is to measure and expose a failure mode so that verifiers can be made reportable and robust. We do not release a turnkey attack service; the generator is a small, reproducible component of a measurement harness, and the paper reports defenses (evidence gating, coverage reporting, abstention accounting) alongside the risk.

IX. Reproducibility and Artifact

The harness, pinned analyzers and rule sets, control set, and corpora are released. Coverage is explicit: Java (a 690-rule ruleset), Python and JavaScript/TypeScript (minimal rule sets) are covered; C is grammar-only; other languages are reported unchecked. A whole-function LLM audit surfaces issues without a ruleset (file context curbs over-flagging), and dependency advisories refresh from the live OSV feed into immutable, dated snapshots. All assets are pinned by hash; the control reproduction (12/18 vulnerable, 1/3 safe) and the Python-pair measurement run fully offline; 144 tests accompany the code. A content-addressed verdict cache makes repeat runs deterministic and free.

X. Conclusion and Future Work

LLM security verifiers track fixes far better than pattern matchers and are precise when they commit (0% fixed-side false positives), but they are not reliably calibrated and are nonetheless silent on about half of their inputs—a weakness that adversarial comments directly amplify, and that composition with deterministic tools does not fix. The actionable reframing is that the bottleneck of a security verifier is coverage, and that its dominant adversarial target is abstention. Future work must (i) scale to hundreds of genuine fixes across at least two languages and three model families with confidence intervals, (ii) evaluate path-level candidates on multi-file revisions, and (iii) treat abstention and coverage as first-class metrics for security agents.

References

[1] Y. Li, T. Zhang, J. Liu et al., "Graph is the verifier: Agentic reinforcement learning for interprocedural vulnerability detection," 2026.

[2] S. Vijay, A. Priyanshu, D. Chapoteau et al., "Antares: Foundation models for agentic vulnerability localization," 2026.

[3] S. Pan, Z. Zheng, J. Zhou et al., "Answer is cheap, show me the evidence! augmenting automated vulnerability assessment with evidence," 2026, iSSTA 2026.

[4] N. Korda and G. Evron, "OpenAnt: LLM-powered vulnerability discovery through code decomposition, adversarial verification, and dynamic testing," 2026.

[5] Anonymous, "A benchmark score that nothing can be concluded from: What the OWASP benchmark measures, and why it is not detection capability," Technical report and artifact (sastaudit); introduces the four-way decomposition, 2026, companion study; source of the instrument adapted here.

[6] C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan, "Lessons from building static analysis tools at Google," Communications of the ACM, 2018.

[7] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?" IEEE Transactions on Software Engineering, 2022.

[8] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023.
[9] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in Proceedings of the 31st USENIX Security Symposium, 2022.
[10] Z. Wu and C. Nita-Rotaru, "ALIBI: Adaptive agentic attacks on LLM-based vulnerability detectors via adversarial code comments," 2026.
[11] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023.
[12] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," in Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2023.
[13] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions," in 2022 IEEE Symposium on Security and Privacy (SP), 2022.
[14] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" in Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2023.
[15] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in Advances in Neural Information Processing Systems 30, 2017.
[16] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in International Conference on Learning Representations (ICLR), 2017.
[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proceedings of the 34th International Conference on Machine Learning (ICML), 2017.
[18] S. Kadavath, T. Conerly, A. Askell et al., "Language models (mostly) know what they know," 2022.
[19] B. Wen, J. Yao, S. Feng, C. Xu, Y. Tsvetkov, B. Howe, and L. L. Wang, "Know your limits: A survey of abstention in large language models," Transactions of the Association for Computational Linguistics, 2025.
[20] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Advances in Neural Information Processing Systems 36, 2023.
[21] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.
[22] P. Wang, L. Li, L. Chen, Z. Cai, D. Sun, Y. Wang et al., "Large language models are not fair evaluators," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024.
[23] A. Panickssery, S. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in Advances in Neural Information Processing Systems 37, 2024.
[24] Y. Zhou, S. Liu, J. K. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in Advances in Neural Information Processing Systems 32, 2019.
[25] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" in 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2025.
[26] OWASP Foundation, "OWASP benchmark project," https://owasp.org/www-project-benchmark/, 2026.
[27] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in 2014 IEEE Symposium on Security and Privacy (SP), 2014.
[28] P. Avgustinov, O. de Moor, M. Peyton Jones, and M. Schäfer, "QL: Object-oriented queries on relational data," in 30th European Conference on Object-Oriented Programming (ECOOP), 2016.
[29] L. Breiman, "Bagging predictors," Machine Learning, 1996.
[30] L. I. Kuncheva and C. J. Whitaker, "Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy," Machine Learning, 2003.
[31] JavaVFC authors, "JavaVFC: A corpus of vulnerability-fixing commits for java," Zenodo record 13731781, 2026, 784 records; screening corpus. A companion audit found ∼18% are genuine security fixes.